# Python Lists & Dictionaries -- Pre-Lab Study Guide (Beginner Edition)

**No Lambdas. No List/Dict Comprehensions. No One-Line Tricks. Every Step Spelled Out.**

This guide assumes you are new to Python. Every piece of code is written the "long way" on purpose -- using plain `for` loops, `if` statements, and regular variables -- so you can see exactly what is happening at each step. Once you're comfortable with this version, you'll be ready to learn the shorter, more advanced Python tricks later.

---

## 1. What is Indexing?

Imagine a list as a row of numbered boxes sitting on a shelf. Python starts counting the boxes from **0**, not from 1. This is one of the very first things that confuses beginners, so read it twice if you need to!

The number attached to each box is called its **index** (plural: indices).

```
numbers = [10, 20, 30, 40, 50]
index:     0   1   2   3   4    <-- counting from the left, starting at 0
index:    -5  -4  -3  -2  -1    <-- counting from the right, starting at -1
```

Notice that the same box has two valid index numbers: for example, the value `50` is both `numbers[4]` (counting from the left) and `numbers[-1]` (counting from the right).

### Forward indexing (counting from the left)

python

```python
numbers = [10, 20, 30, 40, 50]

first_value = numbers[0]
print(first_value)          # Output: 10

third_value = numbers[2]
print(third_value)          # Output: 30
```

### Backward indexing (counting from the right)

python

```
numbers = [10, 20, 30, 40, 50]

last_value = numbers[-1]
print(last_value)               # Output: 50

second_to_last_value = numbers[-2]
print(second_to_last_value)     # Output: 40
```

### Changing a single element using its index

You can also use an index to replace a value inside the list.

python

```
numbers = [10, 20, 30, 40, 50]

numbers[0] = 99
print(numbers)      # Output: [99, 20, 30, 40, 50]
```

### WARNING -- IndexError

If you ask for an index that does not exist, Python stops your program and shows an **IndexError**. This is Python's way of telling you "that box does not exist on the shelf."

python

```
numbers = [10, 20, 30, 40, 50]  # valid indices here are 0, 1, 2, 3, 4

print(numbers[10])  # This line crashes: IndexError: list index out of range
```

**Beginner tip:** Whenever you see `IndexError`, the fix is almost always to double-check your index math. Count the boxes again, starting from 0.

---

## 2. What is Slicing?

Indexing gives you **one** box. Slicing gives you a **whole chunk** of boxes at once, as a brand-new list.

### The format

```
list[start : stop : step]
```

Read this as: *"Start at position* `start`*, stop just before position* `stop`*, and move* `step` *positions at a time."*

| Part | Meaning | If you leave it out |
|---|---|---|
| `start` | Where the slice begins (this box IS included) | Python starts from position 0 |
| `stop` | Where the slice ends (this box is NOT included) | Python goes to the very end |
| `step` | How many positions to move each time | Python assumes 1 (no skipping) |

The most important thing to remember: **`stop` is never included.** `numbers[2:5]` gives you positions 2, 3, and 4 -- but never position 5.

### Basic slicing

python

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Start at index 2, stop before index 5
result = numbers[2:5]
print(result)       # Output: [2, 3, 4]

# No start given -- Python begins at index 0
result = numbers[:4]
print(result)       # Output: [0, 1, 2, 3]

# No stop given -- Python goes all the way to the last item
result = numbers[6:]
print(result)       # Output: [6, 7, 8, 9]

# Neither start nor stop given -- you get every item, as a brand-new list
result = numbers[:]
print(result)       # Output: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
```

### Slicing with a step

python

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# step = 2 means "take one item, then skip one item", repeatedly
result = numbers[::2]
print(result)       # Output: [0, 2, 4, 6, 8]

# Start at index 1, then keep skipping by 2
result = numbers[1::2]
print(result)       # Output: [1, 3, 5, 7, 9]

# A step of -1 tells Python to move backward through the list.
# Combined with no start and no stop, this reverses the whole list.
result = numbers[::-1]
print(result)       # Output: [9, 8, 7, 6, 5, 4, 3, 2, 1, 0]
```

### Slicing with negative indices

Negative numbers count from the right-hand side of the list.

python

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Everything from the 3rd-from-last item onward -- the "last 3" items
result = numbers[-3:]
print(result)       # Output: [7, 8, 9]

# Everything except the last 3 items
result = numbers[:-3]
print(result)       # Output: [0, 1, 2, 3, 4, 5, 6]
```

### IMPORTANT -- Slicing never crashes your program

This is very different from indexing! If your slice numbers go out of range, Python just gives you back as much as it can find -- even an empty list -- instead of raising an error.

python

```python
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

# Asking for way more than exists -- no crash, just an empty list
result = numbers[100:200]
print(result)       # Output: []

# Asking for a negative start that is too far back -- Python just
# clips it to the beginning of the list
result = numbers[-100:3]
print(result)       # Output: [0, 1, 2]
```

**Beginner tip:** Because slicing never crashes, it also never *warns* you if your index math was wrong. If you expected 5 items back and only got 2, that's a signal to re-check your numbers -- Python will not do it for you.

---

## 3. Useful List Techniques

### Reverse a list using slicing

python

```python
numbers = [1, 2, 3, 4, 5]

reversed_numbers = numbers[::-1]
print(reversed_numbers)     # Output: [5, 4, 3, 2, 1]
print(numbers)              # Output: [1, 2, 3, 4, 5]  (the original list is un
```

Slicing always builds a **new** list. The original list, `numbers`, is left exactly as it was.

### Make a real, independent copy of a list

This is one of the most common beginner mistakes in Python, so pay close attention to the difference below.

python

```python
# --- The WRONG way ---
original = [1, 2, 3, 4, 5]

# This does NOT make a copy. 'wrong_copy' just becomes another
# name for the exact same list in memory.
wrong_copy = original
wrong_copy[0] = 99

print(original)     # Output: [99, 2, 3, 4, 5]  <- the original changed too!
```

python

```python
# --- The CORRECT way ---
original = [1, 2, 3, 4, 5]

# Slicing the whole list with [:] builds a brand new, separate list
real_copy = original[:]
real_copy[0] = 99

print(original)     # Output: [1, 2, 3, 4, 5]   <- untouched
print(real_copy)     # Output: [99, 2, 3, 4, 5]
```

### Swap two individual elements

You can swap two values using their index positions on one line, like this:

python

```python
numbers = [10, 20, 30, 40, 50]

numbers[0], numbers[4] = numbers[4], numbers[0]
print(numbers)      # Output: [50, 20, 30, 40, 10]
```

If that line looks confusing at first, here is exactly the same swap, written out the slow, explicit way using a temporary variable. This is what Python is doing behind the scenes:

python

```python
numbers = [10, 20, 30, 40, 50]

# Step 1: remember the value that is about to be overwritten
temporary_value = numbers[0]

# Step 2: copy the value from index 4 into index 0
numbers[0] = numbers[4]

# Step 3: put the remembered value into index 4
numbers[4] = temporary_value

print(numbers)      # Output: [50, 20, 30, 40, 10]
```

Both versions produce the same result. Use whichever one makes more sense to you while you are still learning.

### Insert elements into the middle using slice assignment

python

```python
numbers = [1, 2, 5, 6]

# numbers[2:2] means "insert right here, don't remove anything"
# because the start and stop are the same position
numbers[2:2] = [3, 4]
print(numbers)      # Output: [1, 2, 3, 4, 5, 6]
```

### Delete a range of elements

python

```python
numbers = [1, 2, 3, 4, 5, 6]

# This removes the elements at index 2, 3, and 4 (index 5 is not included)
del numbers[2:5]
print(numbers)      # Output: [1, 2, 6]
```

### Replace a range with different values (the sizes do not have to match)

python

```python
numbers = [1, 2, 3, 4, 5]

# We are replacing 2 elements (index 1 and index 2) with only 1 new element
numbers[1:3] = [99]
print(numbers)      # Output: [1, 99, 4, 5]
```

### Rotate a list to the left

"Rotating left by k" means: take the first k items off the front of the list and move them to the back, keeping everything else in the same order.

python

```python
numbers = [1, 2, 3, 4, 5]
k = 2   # we want to rotate left by 2 positions

# Step 1: grab everything AFTER the cut point
left_part = numbers[k:]
print(left_part)        # [3, 4, 5]

# Step 2: grab everything BEFORE the cut point
right_part = numbers[:k]
print(right_part)       # [1, 2]

# Step 3: glue them back together in the new order
rotated = left_part + right_part
print(rotated)      # Output: [3, 4, 5, 1, 2]
```

### Chunk a list into smaller pieces

"Chunking" means splitting one long list into several smaller lists of a fixed size.

python

```python
```

```python
def chunk_list(lst, n):
    result = []          # this will hold all of our small chunks
    i = 0                # i is our current starting position

    while i < len(lst):
        # Grab n elements starting at position i.
        # Slicing is safe here -- even if there aren't n elements left,
        # Python will just give us however many remain (no crash).
        chunk = lst[i : i + n]

        result.append(chunk)

        i = i + n   # move our starting position forward by n

    return result


numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]

print(chunk_list(numbers, 3))
# Output: [[1, 2, 3], [4, 5, 6], [7, 8, 9]]

print(chunk_list(numbers, 4))
# Output: [[1, 2, 3, 4], [5, 6, 7, 8], [9]]   <- the last chunk is shorter, and
```

### WARNING -- The nested list trap

This is a very common bug for beginners working with grids or boards.

python

```python
# --- The WRONG way ---

# This looks like it creates 3 separate rows, but it does NOT.
# It creates ONE row, and then points to that SAME row 3 times.
grid = [[0, 0, 0]] * 3

grid[0][0] = 1
print(grid)     # Output: [[1, 0, 0], [1, 0, 0], [1, 0, 0]]
# Every row changed, even though we only touched row 0!
```

python

```
# --- The CORRECT way ---

# Build each row as its own, separate list inside a loop
grid = []
for i in range(3):
    row = [0, 0, 0]      # a brand new list is created on every loop pass
    grid.append(row)

grid[0][0] = 1
print(grid)      # Output: [[1, 0, 0], [0, 0, 0], [0, 0, 0]]
# Only row 0 changed, which is what we wanted
```

**Why did that happen?** In the wrong version, `[[0, 0, 0]] * 3` doesn't build 3 different lists -- it takes the *one* list `[0, 0, 0]` and repeats the *same reference* to it three times. Changing it through any one of those three "copies" changes it everywhere, because they are all secretly the same list. Building each row inside a loop avoids this, because a brand new list object gets created every single time.

---

## 4. Practice Problems -- Lists

Use this list for all problems unless told otherwise:

```python
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
```

1. Get the first element and the last element using indexing. Write two different expressions for each (4 expressions total).
2. Get the last 4 elements using slicing -- without calling `len()`.
3. Reverse `lst` using slicing only.
4. Get every 3rd element, starting from index 1.
5. Write a function `split_half(lst)` that splits the list into two halves. It must still work correctly when the list length is odd.
6. Write a function `swap_halves(lst)` that swaps the first half and the second half using slice assignment.
7. Given `lst2 = [0, 1, 2, 3, 4, 5, 6, 7, 8]`, remove the middle 3 elements so the result is `[0, 1, 2, 6, 7, 8]`. Work out which indices they are first, then use `del`.
8. Write a function `rotate_left(lst, k)` that rotates a list left by `k` positions using only slicing (no loop needed).

9. Write a function `chunk_list(lst, n)` that returns a list of sublists, each of size `n`, using a `while` loop.
10. **GA warm-up:** Write a function `single_point_crossover(parent1, parent2)` that picks a random cut point and recombines two parents into two children.

---

## 5. Solutions -- Lists

python

```python
import random

# ------ Problem 1 ------
lst = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]

# First element, two different ways
print(lst[0])
print(lst[-len(lst)])

# Last element, two different ways
print(lst[-1])
print(lst[len(lst) - 1])


# ------ Problem 2 ------
last_four = lst[-4:]
print(last_four)    # Output: [70, 80, 90, 100]


# ------ Problem 3 ------
reversed_lst = lst[::-1]
print(reversed_lst)


# ------ Problem 4 ------
every_third = lst[1::3]
print(every_third)  # Output: [20, 50, 80]


# ------ Problem 5 ------
def split_half(lst):
    # // is floor division -- it divides and rounds down to a whole number
    mid = len(lst) // 2

    left = lst[:mid]
    right = lst[mid:]   # if the list length is odd, the extra item lands here

    return left, right


left_half, right_half = split_half(lst)
print(left_half)    # [10, 20, 30, 40, 50]
print(right_half)   # [60, 70, 80, 90, 100]
```

```python
# ------ Problem 6 ------
def swap_halves(lst):
    mid = len(lst) // 2

    first_half = lst[:mid]
    second_half = lst[mid:]

    lst[:mid] = second_half
    lst[mid:] = first_half

    return lst


numbers = [10, 20, 30, 40, 50, 60, 70, 80, 90, 100]
print(swap_halves(numbers))
# Output: [60, 70, 80, 90, 100, 10, 20, 30, 40, 50]


# ------ Problem 7 ------
# List:  [0, 1, 2, 3, 4, 5, 6, 7, 8]
# Index:  0  1  2  3  4  5  6  7  8
# The middle 3 values (2, 3, 4 -- wait, let's count carefully) are at
# indices 3, 4, 5, which hold the values 3, 4, 5
lst2 = [0, 1, 2, 3, 4, 5, 6, 7, 8]
del lst2[3:6]
print(lst2)     # Output: [0, 1, 2, 6, 7, 8]


# ------ Problem 8 ------
def rotate_left(lst, k):
    # The % (modulo) operator handles the case where k is bigger
    # than the list length, by "wrapping it around"
    k = k % len(lst)

    left_part = lst[k:]
    right_part = lst[:k]

    return left_part + right_part


print(rotate_left([1, 2, 3, 4, 5], 2))     # Output: [3, 4, 5, 1, 2]
```

```python
# ------ Problem 9 ------
def chunk_list(lst, n):
    result = []
    i = 0
    while i < len(lst):
        chunk = lst[i : i + n]
        result.append(chunk)
        i = i + n
    return result


print(chunk_list([1, 2, 3, 4, 5, 6, 7], 3))
# Output: [[1, 2, 3], [4, 5, 6], [7]]


# ------ Problem 10 ------
def single_point_crossover(parent1, parent2):
    # Pick a random cut point. We avoid position 0 and the very last
    # position so that both children actually contain a mix of both parents.
    point = random.randint(1, len(parent1) - 1)

    # Child 1 = the beginning of parent1 + the tail end of parent2
    child1 = parent1[:point] + parent2[point:]

    # Child 2 = the beginning of parent2 + the tail end of parent1
    child2 = parent2[:point] + parent1[point:]

    return child1, child2


p1 = [1, 0, 1, 1, 0]
p2 = [0, 1, 0, 0, 1]
child1, child2 = single_point_crossover(p1, p2)
print('Child 1:', child1)
print('Child 2:', child2)
```

---

## 6. Why Lists Connect Directly to Your GA Lab

A genetic algorithm (GA) represents each candidate solution as a list, usually called a **chromosome**. Almost every GA operation is really just a list operation wearing a fancy name:

| GA Concept | What it actually is in Python |
|---|---|
| Chromosome | A list of genes, e.g. `[1, 0, 1, 1, 0]` |
| Population | A list of chromosomes -- so, a list of lists |
| Fitness evaluation | Looping through a chromosome and counting correct genes |
| Single-point crossover | Slicing at one cut point and gluing two parents back together |
| Two-point crossover | Slicing at two cut points and swapping the middle section |
| Mutation | Changing one element: `chromosome[i] = new_value` |
| Elitism / selection | Slicing the top few individuals from a sorted population list |

Keep this table nearby -- everything in the next section is built from exactly these ideas, just written out step by step.

---

## 7. Genetic Algorithm Skeleton (Step by Step)

This section builds a full, working genetic algorithm. It looks long, but every piece is small. We will build it one function at a time.

```python
```

```python
import random


# ============================================================
# STEP 1 -- Create one individual and a full population
# ============================================================

def create_individual(length, gene_pool):
    # Build one random chromosome by picking genes one at a time.
    # gene_pool is the list of allowed gene values, e.g. [0, 1]
    individual = []
    for i in range(length):
        gene = random.choice(gene_pool)
        individual.append(gene)
    return individual


def create_population(pop_size, length, gene_pool):
    # Build a full population by creating individuals one at a time
    population = []
    for i in range(pop_size):
        individual = create_individual(length, gene_pool)
        population.append(individual)
    return population


# ============================================================
# STEP 2 -- Fitness function
# ============================================================

def fitness(individual, target):
    # "Fitness" just means: how many genes match the target,
    # position by position?
    score = 0
    for i in range(len(individual)):
        if individual[i] == target[i]:
            score = score + 1
    return score


# ============================================================
# STEP 3 -- Selection (Tournament style)
# ============================================================
```

```python
def tournament_selection(population, fitnesses, k=3):
    # Pick k random individuals from the population, and return
    # whichever one of those k has the best fitness score.
    best_individual = None
    best_score = -1

    chosen_indices = random.sample(range(len(population)), k)

    for index in chosen_indices:
        if fitnesses[index] > best_score:
            best_score = fitnesses[index]
            best_individual = population[index]

    return best_individual


# ============================================================
# STEP 4 -- Crossover
# ============================================================

def single_point_crossover(parent1, parent2):
    # Cut both parents at the SAME random point, then swap their tails
    point = random.randint(1, len(parent1) - 1)

    child1 = parent1[:point] + parent2[point:]
    child2 = parent2[:point] + parent1[point:]

    return child1, child2


def two_point_crossover(parent1, parent2):
    # Cut both parents at two random points, then swap the middle section

    # Step 1: pick two different random cut points
    points = random.sample(range(1, len(parent1)), 2)

    # Step 2: sort them so we know which one comes first
    points.sort()
    point_1 = points[0]
    point_2 = points[1]

    # Step 3: build each child out of three pieces:
    #   piece A (before point_1) + piece B (the middle) + piece C (after point_
```

```
    child1 = parent1[:point_1] + parent2[point_1:point_2] + parent1[point_2:]
    child2 = parent2[:point_1] + parent1[point_1:point_2] + parent2[point_2:]

    return child1, child2


# ============================================================
# STEP 5 -- Mutation
# ============================================================

def mutate(individual, mutation_rate, gene_pool):
    # Walk through every gene. For each one, roll a "dice" between
    # 0.0 and 1.0. If it lands below mutation_rate, replace that gene
    # with a new random value from the gene pool.
    for i in range(len(individual)):
        dice_roll = random.random()    # a random float between 0.0 and 1.0
        if dice_roll < mutation_rate:
            individual[i] = random.choice(gene_pool)
    return individual


# ============================================================
# STEP 6 -- Find the best individual in the population
# ============================================================

def find_best(population, target):
    # Start by assuming the first individual is the best...
    best_individual = population[0]
    best_score = fitness(population[0], target)

    # ...then check every other individual to see if it's actually better
    for i in range(1, len(population)):
        current_score = fitness(population[i], target)
        if current_score > best_score:
            best_score = current_score
            best_individual = population[i]

    return best_individual


# ============================================================
# STEP 7 -- Score every individual (returns a list of scores)
# ============================================================
```

```
def score_population(population, target):
    scores = []
    for individual in population:
        score = fitness(individual, target)
        scores.append(score)
    return scores


# ============================================================
# STEP 8 -- Sort the population, best individual first
# ============================================================
#
# We use "selection sort" here -- a simple, beginner-friendly sorting
# method. It repeatedly finds the best remaining individual and moves it
# into its correct position at the front.

def sort_population(population, target):
    # Work on a copy so the original population list is left untouched
    pop_copy = population[:]

    for i in range(len(pop_copy)):
        # Assume position i already holds the best remaining individual...
        best_index = i

        # ...then check every position after it to see if something is better
        for j in range(i + 1, len(pop_copy)):
            score_j = fitness(pop_copy[j], target)
            score_best = fitness(pop_copy[best_index], target)
            if score_j > score_best:
                best_index = j

        # Move the true best individual into position i.
        # (See Section 3 for a fully spelled-out version of a swap
        # using a temporary variable, if this line looks unfamiliar.)
        pop_copy[i], pop_copy[best_index] = pop_copy[best_index], pop_copy[i]

    return pop_copy


# ============================================================
# STEP 9 -- Main GA loop
# ============================================================

def genetic_algorithm(target, pop_size, gene_pool, generations,
```

```
                    mutation_rate=0.01, elitism=2):

    length = len(target)
    population = create_population(pop_size, length, gene_pool)

    for gen in range(generations):

        # Score every individual in the current population
        fitnesses = score_population(population, target)

        # Elitism: the top few individuals survive unchanged into
        # the next generation, so we never lose our best solution
        ranked_population = sort_population(population, target)
        new_population = ranked_population[:elitism]

        # Fill up the rest of the new population with children
        while len(new_population) < pop_size:
            parent1 = tournament_selection(population, fitnesses)
            parent2 = tournament_selection(population, fitnesses)

            child1, child2 = single_point_crossover(parent1, parent2)

            child1 = mutate(child1, mutation_rate, gene_pool)
            child2 = mutate(child2, mutation_rate, gene_pool)

            new_population.append(child1)
            new_population.append(child2)

        # We might have added one extra child, so trim back to pop_size
        population = new_population[:pop_size]

        # Report progress for this generation
        best = find_best(population, target)
        best_score = fitness(best, target)
        print('Generation', gen, '| Best score:', best_score, '| Best:', best)

        # Stop early if we've found a perfect match
        if best_score == length:
            print('Solved at generation', gen, '!')
            break

    return find_best(population, target)
```

```python
# Run it!
target = [1, 0, 1, 1, 0, 1, 0, 0, 1, 1]
result = genetic_algorithm(
    target=target,
    pop_size=50,
    gene_pool=[0, 1],
    generations=200
)
print('Final answer:', result)
```

---

## 8. Dictionaries -- The Basics

A **dictionary** stores information as **key --> value** pairs. This is different from a list, where you find things by their position number (index). In a dictionary, you find things by **name** (the key).

python

```python
student = {
    'name': 'Lev',
    'age': 21,
    'grade': 'A'
}

# Access a value by giving its key
name = student['name']
age = student['age']
print(name)     # Output: Lev
print(age)      # Output: 21
```

### Crash vs. safe access

python

```python
student = {'name': 'Lev', 'age': 21}

# This line would CRASH, because the key 'gpa' does not exist:
# print(student['gpa'])           # KeyError: 'gpa'

# .get() is a safer way to look something up.
# If the key is missing, it returns None instead of crashing.
gpa = student.get('gpa')
print(gpa)          # Output: None

# You can also give .get() a default value to use if the key is missing
gpa = student.get('gpa', 0.0)
print(gpa)          # Output: 0.0
```

### WARNING -- You CANNOT slice a dictionary

Slicing only works on things that have an order, like lists and strings. Dictionaries do not guarantee a fixed position for each item the way a list does, so slicing simply is not allowed.

python

```python
d = {'a': 1, 'b': 2, 'c': 3}

# This line would CRASH:
# print(d[1:3])       # TypeError: unhashable type: 'slice'

# The workaround: convert the dictionary's items into a list first,
# and THEN slice that list.
items = list(d.items())     # [('a', 1), ('b', 2), ('c', 3)]

first_two = items[:2]
print(first_two)    # Output: [('a', 1), ('b', 2)]

last_two = items[-2:]
print(last_two)     # Output: [('b', 2), ('c', 3)]
```

### Adding, updating, and deleting entries

python

```python
```

```python
student = {'name': 'Lev', 'age': 21}

# Add a brand new key
student['grade'] = 'A'
print(student)      # {'name': 'Lev', 'age': 21, 'grade': 'A'}

# Update the value of a key that already exists
student['age'] = 22
print(student)      # {'name': 'Lev', 'age': 22, 'grade': 'A'}

# Delete a key entirely
del student['grade']
print(student)      # {'name': 'Lev', 'age': 22}
```

### Looping through a dictionary

python

```python
d = {'a': 1, 'b': 2, 'c': 3}

# Looping this way gives you just the keys
for key in d:
    print(key)

# Looping this way gives you both the key and the value together
for key, value in d.items():
    print(key, '->', value)

# Looping this way gives you just the values
for value in d.values():
    print(value)
```

### Merging two dictionaries

There is a shorter way to do this using the `|` operator in newer versions of Python, but the beginner-friendly way below works in every version of Python 3 and makes it very clear what is happening: whichever dictionary you call `.update()` with LAST is the one that wins on any shared keys.

python

```python
d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 3}

merged = {}
merged.update(d1)   # first, copy in everything from d1
merged.update(d2)   # then, copy in everything from d2 (overwriting shared keys

print(merged)       # Output: {'a': 1, 'b': 99, 'c': 3}
```

---

## 9. Dictionary Techniques (No Comprehensions)

### Invert a dictionary -- swap keys and values

python

```python
def invert_dict(d):
    inverted = {}
    for key, value in d.items():
        # the OLD value becomes the NEW key, and vice versa
        inverted[value] = key
    return inverted


original = {'a': 1, 'b': 2, 'c': 3}
flipped = invert_dict(original)
print(flipped)      # Output: {1: 'a', 2: 'b', 3: 'c'}
```

### Build a dictionary from two separate lists

python

```python
def build_dict_from_lists(keys, values):
    result = {}
    for i in range(len(keys)):
        result[keys[i]] = values[i]
    return result


key_list = ['x', 'y', 'z']
value_list = [10, 20, 30]
d = build_dict_from_lists(key_list, value_list)
print(d)    # Output: {'x': 10, 'y': 20, 'z': 30}
```

### Filter a dictionary -- keep only entries that pass a condition

python

```python
def filter_dict(d, min_value):
    filtered = {}
    for key, value in d.items():
        if value > min_value:
            filtered[key] = value
    return filtered


d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
result = filter_dict(d, 4)
print(result)   # Output: {'a': 5, 'c': 8, 'e': 9}
```

### Sort a dictionary by value, largest first

We use the same simple "selection sort" idea from Section 7 here.

python

```python
def sort_dict_by_value(d):
    # Step 1: collect every key-value pair into a plain list,
    # where each pair is stored as a small [key, value] list
    items = []
    for key, value in d.items():
        items.append([key, value])

    # Step 2: sort those pairs by value, largest first, using
    # selection sort (repeatedly find the largest remaining pair)
    for i in range(len(items)):
        best_index = i
        for j in range(i + 1, len(items)):
            if items[j][1] > items[best_index][1]:
                best_index = j
        items[i], items[best_index] = items[best_index], items[i]

    # Step 3: rebuild a dictionary from the now-sorted pairs
    sorted_dict = {}
    for pair in items:
        key = pair[0]
        value = pair[1]
        sorted_dict[key] = value

    return sorted_dict


d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
print(sort_dict_by_value(d))
# Output: {'e': 9, 'c': 8, 'a': 5, 'b': 2, 'd': 1}
```

### Count how many times each item appears in a list

python

```python
def count_occurrences(lst):
    counts = {}
    for item in lst:
        if item in counts:
            counts[item] = counts[item] + 1
        else:
            counts[item] = 1
    return counts


data = ['cat', 'dog', 'cat', 'bird', 'dog', 'cat']
print(count_occurrences(data))
# Output: {'cat': 3, 'dog': 2, 'bird': 1}
```

---

## 10. Practice Problems -- Dictionaries

```python
d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}
```

1. Get the value for key `'c'` -- two different ways (one that can crash, one that cannot).
2. Try to access a key `'z'` that does not exist -- once in a way that crashes, and once in a way that is safe.
3. Get the first 2 and the last 2 key-value pairs, as two separate lists.
4. Write a function `invert_dict(d)` that swaps keys and values using a `for` loop.
5. Write a function `filter_dict(d, min_val)` that keeps only the entries where the value is greater than `min_val`.
6. Write a function `sort_by_value(d)` that returns a new dictionary sorted by value, largest first.
7. Given `keys = ['x', 'y', 'z']` and `values = [10, 20, 30]`, build a dictionary using a `for` loop.
8. Merge `d1 = {'a': 1, 'b': 2}` and `d2 = {'b': 99, 'c': 3}` so that `d2` wins on any shared keys.

### Solutions

```python
```

```python
# ------ Problem 1 ------
d = {'a': 5, 'b': 2, 'c': 8, 'd': 1, 'e': 9}

value1 = d['c']         # direct access -- would crash if 'c' didn't exist
value2 = d.get('c')     # safe access -- would return None instead of crashing
print(value1)           # 8
print(value2)           # 8


# ------ Problem 2 ------
safe_result = d.get('z')            # returns None, no crash
print(safe_result)                  # None

safe_with_default = d.get('z', 0)
print(safe_with_default)            # 0

# Uncomment the next line to see the crash for yourself:
# print(d['z'])                     # KeyError: 'z'


# ------ Problem 3 ------
all_items = list(d.items())
print(all_items[:2])    # the first 2 pairs
print(all_items[-2:])   # the last 2 pairs


# ------ Problem 4 ------
def invert_dict(d):
    inverted = {}
    for key, value in d.items():
        inverted[value] = key
    return inverted


print(invert_dict(d))   # {5: 'a', 2: 'b', 8: 'c', 1: 'd', 9: 'e'}


# ------ Problem 5 ------
def filter_dict(d, min_val):
    result = {}
    for key, value in d.items():
        if value > min_val:
            result[key] = value
```

```
    return result


print(filter_dict(d, 4))    # {'a': 5, 'c': 8, 'e': 9}


# ------ Problem 6 ------
def sort_by_value(d):
    items = []
    for key, value in d.items():
        items.append([key, value])

    for i in range(len(items)):
        best_index = i
        for j in range(i + 1, len(items)):
            if items[j][1] > items[best_index][1]:
                best_index = j
        items[i], items[best_index] = items[best_index], items[i]

    sorted_dict = {}
    for pair in items:
        sorted_dict[pair[0]] = pair[1]
    return sorted_dict


print(sort_by_value(d))     # {'e': 9, 'c': 8, 'a': 5, 'b': 2, 'd': 1}


# ------ Problem 7 ------
def build_dict_from_lists(keys, values):
    result = {}
    for i in range(len(keys)):
        result[keys[i]] = values[i]
    return result


keys = ['x', 'y', 'z']
values = [10, 20, 30]
print(build_dict_from_lists(keys, values))  # {'x': 10, 'y': 20, 'z': 30}


# ------ Problem 8 ------
d1 = {'a': 1, 'b': 2}
d2 = {'b': 99, 'c': 3}
```

```python
merged = {}
merged.update(d1)
merged.update(d2)   # d2 overwrites 'b'
print(merged)       # {'a': 1, 'b': 99, 'c': 3}
```

---

## 11. List + Dictionary Combined -- The GA-Ready Pattern

Once a genetic algorithm gets a bit more advanced, each individual is usually stored as a **dictionary** that holds both its genes AND its fitness score together, instead of tracking them in two separate lists.

```python
population = [
    {'genes': [1, 0, 1, 1, 0], 'fitness': 3},
    {'genes': [0, 1, 1, 0, 0], 'fitness': 2},
    {'genes': [1, 1, 1, 1, 1], 'fitness': 5},
]
```

`population` here is a **list of dictionaries.** That means you can combine indexing and key-lookups together:

```python
# Get the second individual (index 1) -- this is a whole dictionary
second = population[1]
print(second)              # {'genes': [0, 1, 1, 0, 0], 'fitness': 2}

# Get just that individual's genes
genes = population[1]['genes']
print(genes)               # [0, 1, 1, 0, 0]

# Get one specific gene inside that individual (the 3rd gene, index 2)
third_gene = population[1]['genes'][2]
print(third_gene)          # 1
```

### Problem 1 -- Extract all gene lists

```python
```

```
def get_all_genes(population):
    gene_lists = []
    for individual in population:
        gene_lists.append(individual['genes'])
    return gene_lists


genes = get_all_genes(population)
print(genes)
# Output: [[1, 0, 1, 1, 0], [0, 1, 1, 0, 0], [1, 1, 1, 1, 1]]
```

### Problem 2 -- Sort the population by fitness, best first

python

```
def sort_population_by_fitness(population):
    pop_copy = population[:]    # work on a copy, leave the original alone

    for i in range(len(pop_copy)):
        best_index = i
        for j in range(i + 1, len(pop_copy)):
            if pop_copy[j]['fitness'] > pop_copy[best_index]['fitness']:
                best_index = j
        pop_copy[i], pop_copy[best_index] = pop_copy[best_index], pop_copy[i]

    return pop_copy


sorted_pop = sort_population_by_fitness(population)
for individual in sorted_pop:
    print(individual)
```

### Problem 3 -- Get the top 2 fittest individuals

python

```
def get_top_k(population, k):
    sorted_pop = sort_population_by_fitness(population)
    return sorted_pop[:k]


top_two = get_top_k(population, 2)
for individual in top_two:
    print(individual)
```

### Problem 4 -- Compute average fitness

python

```
def average_fitness(population):
    total = 0
    for individual in population:
        total = total + individual['fitness']
    average = total / len(population)
    return average


avg = average_fitness(population)
print(avg)      # Output: 3.333...
```

### Problem 5 -- Build a population of dictionaries from raw gene lists

python

```python
def count_ones(genes):
    # A simple fitness function: it just counts how many 1s are present
    count = 0
    for gene in genes:
        if gene == 1:
            count = count + 1
    return count


def build_population_from_genes(gene_lists):
    population = []
    for genes in gene_lists:
        score = count_ones(genes)
        individual = {'genes': genes, 'fitness': score}
        population.append(individual)
    return population


raw_genes = [
    [1, 0, 1],
    [0, 0, 1],
    [1, 1, 1]
]
pop = build_population_from_genes(raw_genes)
print(pop)
# [{'genes': [1, 0, 1], 'fitness': 2}, {'genes': [0, 0, 1], 'fitness': 1}, {'ge
```

## Problem 6 -- Group individuals by fitness score

python

```
def group_by_fitness(population):
    groups = {}
    for individual in population:
        score = individual['fitness']
        if score not in groups:
            groups[score] = []
        groups[score].append(individual)
    return groups


grouped = group_by_fitness(population)
for score, members in grouped.items():
    print('Fitness', score, '->', members)
```

### Problem 7 -- Crossover an entire population of dictionaries

python

```python
import random


def crossover_population(population):
    children = []
    i = 0

    while i < len(population) - 1:
        parent1 = population[i]
        parent2 = population[i + 1]

        genes1 = parent1['genes']
        genes2 = parent2['genes']

        point = random.randint(1, len(genes1) - 1)

        child1_genes = genes1[:point] + genes2[point:]
        child2_genes = genes2[:point] + genes1[point:]

        # We don't know the fitness of a child yet, so we mark it as None
        # until it gets evaluated later
        child1 = {'genes': child1_genes, 'fitness': None}
        child2 = {'genes': child2_genes, 'fitness': None}

        children.append(child1)
        children.append(child2)

        i = i + 2   # move ahead to the next pair of parents

    return children


result = crossover_population(population)
for child in result:
    print(child)
```

---

## 12. Quick Reference Cheat Sheet

### Lists

```
lst[i]              get one element by index    -- crashes on a bad index
```

```
(IndexError)
lst[i:j]              get a chunk from i to j     -- never crashes
lst[i:j:k]            every kth element from i to j
lst[::-1]             reverse the entire list
lst[:]                make an independent copy
lst[a:b] = [...]      replace a range of elements (the new list can be a
different length)
del lst[a:b]          remove a range of elements
```

### Dictionaries

```
d[key]               get a value by its key       -- crashes with KeyError if
the key is missing
d.get(key)           safe access                  -- returns None if the key
is missing
d.get(key, default)  safe access                  -- returns your chosen
default if missing
d[1:3]               INVALID                      -- TypeError, dictionaries
cannot be sliced
list(d.items())[:n]  workaround: convert to a list first, then slice that
list
d.update(d2)         merge d2 into d              -- d2 wins on any shared
keys
del d[key]           remove one key-value pair
```

### List of Dictionaries (the GA pattern)

```
population[i]                 one individual (a dictionary)
population[i]['genes']        that individual's list of genes
population[i]['genes'][j]     one specific gene inside that individual
population[:k]                the top k individuals (after sorting)
population[i]['fitness']      that individual's fitness score
```